

JOHANNES KEPLER
UNIVERSITY LINZ

ASSIGNMENT 1: FOUNDATIONS



Lukas Steu

JOHANNES KEPLER
UNIVERSITY LINZ

Task 1

Exercise 1: Empirical Joint Distribution

```python
from pathlib import Path
import numpy as np

# find the CSV from the notebook or project directory
path = next(p for p in (Path("students.csv"), Path("3rd_Semester/UE_Probabilistic_Models/UE_Ex1/students.csv")) if p.is_file())

# read the observations, skipping the header
data = np.loadtxt(path, delimiter=",", skiprows=1, dtype=str)
indices = np.array([[int(value[1:]) for value in row] for row in data])

# convert category labels such as d0 and g1 to integers
indices[:, 2] -= 1

# allocate one count for each combination in axis order D,I,G,S,L
counts = np.zeros((2, 2, 3, 2, 2), dtype=int)

# count every observation, including repeated combinations
np.add.at(counts, tuple(indices.T), 1)

# estimate each joint probability as its count divided by the number of observations
P_emp = counts / len(data)
assert counts.sum() == len(data) and np.isclose(P_emp.sum(), 1)

# check that every observation was counted and probabilities sum to one
# report sample size, array shape, and 48 minus one normalization constraint
print(f"Observations: {len(data)}; shape: {P_emp.shape}; free parameters: {P_emp.size - 1}")
# display the complete empirical joint distribution
#print(P_emp)
```

[1] ✓ 0.6s Python

```
Observations: 1182; shape: (2, 2, 3, 2, 2); free parameters: 47
```

Task 2

Exercise 2: Probabilistic Queries

```python
# sum over D,I,G,S while fixing a strong letter L=l1
p1 = P_emp[:, :, :, :, 1].sum()

# divide P(I=i1,S=s1) by P(S=s1), summing over all other variables
p2 = P_emp[:, 1, :, 1, :].sum() / P_emp[:, :, :, 1, :].sum()

# divide P(D=d1,G=g1) by P(G=g1); grade A uses index 0
p3 = P_emp[1, :, 0, :, :].sum() / P_emp[:, :, 0, :, :].sum()

# sum over D,G to obtain P(I=i1,S=s1,L=l1)
numerator = P_emp[:, 1, :, 1, 1].sum()

# sum over D,I,G to obtain P(S=s1,L=l1)
denominator = P_emp[:, :, :, 1, 1].sum()

# normalize the joint probability by the probability of the evidence
p4 = numerator / denominator

# the probability of a strong recommendation letter
print(f"P(L=l1) = {p1:.6f} ({p1:.2%})")
# high interest given a high SAT score
print(f"P(I=i1 | S=s1) = {p2:.6f} ({p2:.2%})")
# a hard course given an A grade
print(f"P(D=d1 | G=g1) = {p3:.6f} ({p3:.2%})")
# high interest given a strong letter and a high SAT score
print(f"P(I=i1 | L=l1,S=s1) = {p4:.6f} ({p4:.2%})")
# intermediate enumeration result
print(f"Last query: numerator = {numerator:.6f}; denominator = {denominator:.6f}")
```

[3] ✓ 0.0s Python

```
P(L=l1) = 0.483926 (48.39%)
P(I=i1 | S=s1) = 0.778243 (77.82%)
P(D=d1 | G=g1) = 0.047146 (4.71%)
P(I=i1 | L=l1,S=s1) = 0.888889 (88.89%)
Last query: numerator = 0.142132; denominator = 0.159898
```

Task 3

Exercise 3: Independence in the empirical distribution

```python
# compare a joint table P(X,Y) with the product of its two marginals
def deviation(table):
    # normalize the table to obtain the joint distribution, conditional on any fixed state
    joint = table / table.sum()
    # keep row and column dimensions so broadcasting produces all marginal products
    product = joint.sum(axis=1, keepdims=True) * joint.sum(axis=0, keepdims=True)
    # return the largest absolute difference across all combinations
    return np.abs(joint - product).max()

# sum over G,S,L to obtain the joint table for D and I
error1 = deviation(P_emp.sum(axis=(2, 3, 4)))
# for each I, sum over L and group D,G into six row states; columns represent S
error2 = max(deviation(P_emp[:, i, :, :, :].sum(axis=3).reshape(6, 2)) for i in range(2))
# for each G, group D,I,S into eight row states; columns represent L
error3 = max(deviation(P_emp[:, :, g, :, :].reshape(8, 2)) for g in range(3))
# collect the three claims and maximum deviations over all relevant states
results = [("D independent of I", error1), ("S independent of (D,G) given I", error2), ("L independent of (D,I,S) given G", error3)]
# iterate over the results to display each claim
for claim, error in results:
    # allow only floating-point rounding; this tolerance is not a statistical significance test
    print(f"{claim}: {error < 1e-12}; maximum absolute difference = {error:.8f}")
```

```
D independent of I: False; maximum absolute difference = 0.06496990
S independent of (D,G) given I: False; maximum absolute difference = 0.00822597
L independent of (D,I,S) given G: False; maximum absolute difference = 0.01088610
```

Task 4

Exercise 4: Factorisation and graphical model

```
# count all joint states and remove the single normalization constraint
full_parameters = np.prod(P_emp.shape) - 1
# count free child probabilities separately for every parent configuration
model_parameters = (2 - 1) + (2 - 1) + 2 * 2 * (3 - 1) + 2 * (2 - 1) + 3 * (2 - 1)
# report the parameter counts for both models
print(f"Full joint: {full_parameters}; factorised model: {model_parameters}")
```

[5] ✓ 0.0s Python

```
Full joint: 47; factorised model: 15
```

Task 5 (Bonus 1)

Exercise 5: Model-based full joint distribution

```python
# normalize each child distribution along the last axis
def normalize(table):
    # count observations for each parent configuration while keeping the child axis
    total = table.sum(axis=-1, keepdims=True)
    # divide where counts are positive and otherwise keep a uniform child distribution
    return np.divide(table, total, out=np.full(table.shape, 1 / table.shape[-1]), where=total > 0)

# estimate difficulty by summing over all other variables
pD = normalize(counts.sum(axis=(1, 2, 3, 4)))

# estimate interest by summing over all other variables
pI = normalize(counts.sum(axis=(0, 2, 3, 4)))

# estimate grade given difficulty and interest by summing over SAT and letter
pG = normalize(counts.sum(axis=(3, 4)))

# estimate SAT given interest by summing over difficulty, grade and letter
pS = normalize(counts.sum(axis=(0, 2, 4)))

# estimate letter given grade by summing over difficulty, interest and SAT
pL = normalize(counts.sum(axis=(0, 1, 3)))

# count missing parent configurations for grade, SAT and letter
missing = [int((table.sum(axis=-1) == 0).sum()) for table in (counts.sum(axis=(3, 4)), counts.sum(axis=(0, 2, 4)), counts.sum(axis=(0, 1, 3)))]

# report how many uniform fallback distributions were needed
print(f"Missing parent configurations for G, S, L: {missing}")

# allocate the joint model array in axis order D,I,G,S,L
P_model = np.zeros_like(P_emp)

# visit every combination of the five variables
for d, i, g, s, l in np.ndindex(P_model.shape):
    # multiply the five local probabilities for this combination
    P_model[d, i, g, s, l] = pD[d] * pI[i] * pG[d, i, g] * pS[i, s] * pL[g, l]

# check nonnegative entries and normalization of the full distribution
assert np.all(P_model >= 0) and np.isclose(P_model.sum(), 1)
# iterate over all estimated local probability tables
for name, table in [("P(D)", pD), ("P(I)", pI), ("P(G|D,I)", pG), ("P(S|I)", pS), ("P(L|G)", pL)]:
    # display each table with six decimal places
    print(f"{name}:\n{np.array2string(table, precision=6)}")
```

[4] ✓ 0.0s Python

```
Missing parent configurations for G, S, L: [1, 0, 0]
P(D):
[0.675973 0.324027]
P(I):
[0.799492 0.200508]
P(G|D,I):
[[[0.291815 0.3879   0.320285]
  [0.92827  0.07173  0.      ]]

 [[0.049608 0.234987 0.715405]
  [0.333333 0.333333 0.333333]]]
P(S|I):
[[0.943915 0.056085]
 [0.21519  0.78481 ]]
P(L|G):
[[0.064516 0.935484]
 [0.409231 0.590769]
 [0.993392 0.006608]]
```

Task 6 (Bonus 2)

Exercise 6: Compare the two distributions

```python
# compare both full joint distributions cell by cell
print(f"Maximum difference: {np.abs(P_emp - P_model).max():.8f}; total variation: {0.5 * np.abs(P_emp - P_model).sum():.8f}")
# repeat the difficulty-interest independence check with the model
model_error1 = deviation(P_model.sum(axis=(2, 3, 4)))
# repeat the conditional SAT check for both interest states
model_error2 = max(deviation(P_model[:, i, :, :, :].sum(axis=3).reshape(6, 2)) for i in range(2))
# repeat the conditional letter check for all three grades
model_error3 = max(deviation(P_model[:, :, g, :, :].reshape(8, 2)) for g in range(3))
# pair empirical and model deviations for each claim
comparison = [("D independent of I", error1, model_error1), ("S independent of (D,G) given I", error2, model_error2), ("L independent of (D,I,S) given G", error3,
# iterate over all three comparisons
for claim, empirical, model in comparison:
    # report deviations and whether the model satisfies independence within rounding tolerance
    print(f"{claim}: empirical={empirical:.8f}; model={model:.2e}; model independent={model < 1e-12}")
```

[8] ✓ 0.0s Python

```
Maximum difference: 0.04533959; total variation: 0.13539138
D independent of I: empirical=0.06496990; model=2.78e-17; model independent=True
S independent of (D,G) given I: empirical=0.00822597; model=1.11e-16; model independent=True
L independent of (D,I,S) given G: empirical=0.01088610; model=1.67e-16; model independent=True
```